



From static tasks to dynamic reasoning: a characterization framework and study of large language models in next-generation cybersecurity automation

Hanbo Yu¹ · Shahrear Iqbal¹ · Euclides Carlos Pinto Neto¹ · Scott Buffett¹ · Adrian Taylor²

Received: 3 March 2026 / Accepted: 12 August 2026
© Crown 2026

Abstract

Large Language Models (LLMs) are increasingly applied in cybersecurity, but most existing industry use cases focus on static, one-shot tasks such as classification, entity extraction, or summarization. While effective in narrow contexts, these applications fail to capture the complexity of real-world cybersecurity workflows, which often unfold over time, involve evolving inputs, and require multi-step reasoning. In this paper, we shift the focus toward dynamic cyber tasks—problems that demand context awareness, tool interaction, and adaptive decision-making. Our main goal is to define, analyze, and investigate the role of LLMs in automating these dynamic tasks. To achieve this, we introduce a characterization framework that profiles dynamic cyber tasks along four complementary dimensions: operational goal, knowledge grounding, collaboration mode, and cognitive complexity. We survey recent LLM-based systems across seven core domains: threat intelligence, data privacy and security, vulnerability detection, malware detection, intrusion detection, incident response and red teaming automation. Our analysis shows that current systems remain limited by privacy and deployment constraints, stale or incomplete threat knowledge, weak validation of feedback-driven actions, and insufficient evidence of operational benefit. We identify the need for privacy-aware deployment, timely retrieval and knowledge maintenance for emerging threats, process-level evaluation tied to measurable security outcomes, and human oversight within controlled and hybrid automation workflows. These findings clarify where LLMs can provide practical value and where conventional or hybrid approaches may remain more suitable.

Keywords Knowledge management · Cybersecurity · Artificial intelligence · Knowledge extraction

1 Introduction

Large Language Models (LLMs) have shown significant potential in automating complex tasks across a range of high-impact domains, including healthcare, law, and finance [1–3].

As their reasoning, planning, and interaction capabilities rapidly progress, there is growing interest in exploring their applicability to cybersecurity—an area that faces increasing pressure to respond to sophisticated threats [4]. Cyber operations often require rapid decision-making under uncertainty, making them promising tools for intelligent automation.

While existing work has applied LLMs to static cyber tasks such as entity extraction and CVE classification, there is an increasing interest in the research community to use LLMs in real-world security workflows that are often very complex. Many workflows unfold over time, span multiple systems, and depend heavily on evolving context. These are what we refer to as dynamic cybersecurity automation tasks—processes that go beyond single-shot predictions and instead involve reasoning, interaction, and multi-step coordination to accomplish a security objective.

These types of tasks present a unique opportunity. As LLMs become more capable of holding context, processing diverse inputs, and coordinating complex operations, they

✉ Shahrear Iqbal
Shahrear.Iqbal@nrc-cnrc.gc.ca

Hanbo Yu
Hanbo.Yu@nrc-cnrc.gc.ca

Euclides Carlos Pinto Neto
EuclidesCarlos.PintoNeto@nrc-cnrc.gc.ca

Scott Buffett
Scott.Buffett@nrc-cnrc.gc.ca

Adrian Taylor
Adrian.Taylor@forces.gc.ca

¹ National Research Council Canada, Fredericton, NB, Canada

² Defence Research and Development Canada (DRDC),
Ottawa, ON, Canada

are increasingly suited to take on higher-level responsibilities in security workflows—from investigating alerts and correlating logs to generating mitigation strategies and simulating attacker behavior [5]. Supporting these tasks with LLMs could help reduce analyst fatigue, improve response times, and bring greater consistency and scalability to cybersecurity operations.

This paper takes a step in that direction. Our goal is to define what dynamic cyber tasks are, explain why they matter, and explore how LLMs can help automate them. We introduce a characterization framework to make sense of the space, analyze how current systems are approaching these challenges, and identify areas where more progress is needed. Ultimately, we provide a foundation for future work that moves beyond static outputs and toward more intelligent, flexible, and collaborative cybersecurity automation.

The main contributions of this research are:

- *A characterization framework for dynamic cyber automation tasks:* Current research on LLMs in cybersecurity is fragmented and often focused on isolated tasks. There is no clear structure to understand or compare how LLMs can automate more complex, multi-step cyber operations. Considering operational goals, knowledge grounding requirements, collaboration modes, and cognitive complexity, our framework provides a common language for describing and comparing dynamic tasks.
- *An analysis of state-of-the-art works focused on the application of LLMs for dynamic cyber automation tasks:* We focus on major cybersecurity areas, including threat intelligence, data privacy & security, vulnerability detection, malware detection, intrusion detection, incident response, and red teaming automation. The selection of domains in this study is inspired by existing literature. The review presented in [6] provided the initial foundation of key areas where pre-trained language models have been actively applied, including threat intelligence, malware detection, vulnerability detection, and data privacy and security. Building on this, we expand the scope to better capture the demands of dynamic cybersecurity operations. In particular, the growing focus on proactive adversarial testing and red team simulation in LLM safety discussions [7] motivated the explicit inclusion of red teaming automation. Similarly, recent work [8] applying LLMs to incident response planning and coordination emphasized the need to recognize incident response as a distinct domain.
- *The identification of gaps and future opportunities and directions:* We present a comprehensive evaluation of immediate research directions and critical problems to be solved in the next few years. The list includes the development of domain-specific reasoning agents, human-AI collaboration, benchmark development direction and

multi-agent collaboration in the context of dynamic cyber tasks. We also analyze aspects such as the lack of specialized LLMs tuned for complex, domain-specific cybersecurity workflows; the limited integration between LLMs and human analysts for collaborative decision-making; the absence of realistic benchmarks that evaluate multi-step reasoning, tool usage, and adaptability; and the fragmented exploration of how multiple LLM agents could coordinate across different phases of cyber operations in dynamic environments.

This paper is organized as follows: Sect. 2 presents the related works. After that, Sect. 3 presents the characterization of dynamic cyber automation tasks. Then, Sect. 4 describes the LLM applications for dynamic cyber automation tasks. Finally, Sects. 5 and 6 present future directions and the conclusion for this research, respectively.

2 Related works

This section analyzes existing works that present a similar objective to this research. We present two different perspectives: (i) the relevance of Large Language Models (LLMs) in the context of cybersecurity and (ii) surveys that focus on complementary areas. This analysis highlights the important insights of the state-of-the-art solutions, while identifying the unique contributions of our study.

2.1 Background on large language models and their applications in cybersecurity

The development of LLMs has been one of the most revolutionary advancements in artificial intelligence research. Early Deep Learning (DL) models, such as Convolutional Networks (CNNs) [9] and Recurrent Networks (RNNs) [10], enabled breakthroughs in vision and speech but struggled with long-range linguistic dependencies and limited scalability [11]. The introduction of the transformer architecture fundamentally changed the rule by replacing recurrence with self-attention, which allows models to process entire sequences in parallel and to capture contextual relations across tokens. Building on this foundation, pre-training on massive text corpora followed by task-specific fine-tuning became the dominant paradigm. Contextual word-embedding models such as ELMo [12] and BERT [13] demonstrated that general-purpose representations could outperform handcrafted features across different language tasks, while T5 [14] unified multiple objectives into a single text-to-text framework. In parallel, the GPT series [15] established autoregressive generation as a powerful alternative to masked-language modeling, which leads to the development of instruction-tuned and reinforcement-aligned variants such

as InstructGPT [16] and GPT-4 [17]. The open-source efforts, such as PaLM [18], LLaMA [19], Mistral [20], and the multimodal Gemini family [21], extended this capability to multilingual reasoning, code understanding, and multimodal synthesis. These advances collectively transformed LLMs from text predictors into interactive reasoning systems capable of dialogue, planning, and contextual decision-making.

As LLMs matured, researchers began exploring their potential within cybersecurity, a field characterized by vast unstructured data and high contextual complexity. Early applications adapted pre-trained encoders like BERT and SciBERT to process vulnerability reports, threat advisories, and phishing datasets [22, 23]. These models automatically extracted entities, mapped vulnerabilities to CWE categories, and summarized incident narratives, which shows that linguistic representations could capture domain-specific semantics beyond keyword matching. The introduction of large generative models such as GPT-3 started a new round of research. Analysts adopted few-shot prompting to generate exploit explanations, paraphrase security documentation, or produce synthetic attack data. Tools like VulGPT [24] and PentestGPT [25] began emulating penetration-testing reasoning, while GENTTP [26] achieved zero-shot extraction of tactics, techniques, and procedures (TTPs) from threat-intelligence reports.

Building upon these foundations, recent work has begun integrating LLMs directly into cybersecurity workflows. Zangana et al. [27] provide a broad overview of how LLMs are used in cybersecurity, showing how they have moved from basic automation to more advanced and intelligent applications. Their chapter discusses key operational areas such as threat detection, vulnerability management, and incident response, and emphasizes the potential of LLMs to support context-aware decision-making. However, their discussion remains at a conceptual level and does not explicitly define or structure dynamic cyber tasks. Specifically, they neither differentiate between static and dynamic workflows, nor analyze the multi-step reasoning processes, tool interactions, or adaptive behaviors that characterize dynamic tasks. Moreover, while they mention future directions like synthetic data usage and blockchain integration, they do not present a detailed technical breakdown of how LLMs could be operationalized to manage complex, evolving cybersecurity scenarios. Our work extends this discussion by explicitly defining dynamic cyber tasks, organizing them into a structured characterization framework, and examining how LLMs are being used to automate them, to address a significant gap left by existing studies.

A related effort by Sultana et al. [28] proposes a conceptual framework for benchmarking LLMs in cybersecurity, introducing an initial distinction between static and dynamic tasks. While this classification provides a useful starting point, practical studies on dynamic applications remain unex-

plored. As LLMs evolve into agentic systems capable of interacting with tools and environments in real time, it becomes important to evaluate their performance on dynamic tasks for developing trustworthy cyber-operations automation. Recent research [29, 30] continues to focus primarily on static tasks such as log parsing, named entity recognition, or malware classification tasks that are valuable but involve limited contextual reasoning and are typically formulated as single-turn inference problems. However, real-world cyber defense is inherently dynamic. It unfolds over time, requires multistep reasoning, and demands the ability to respond to incomplete or ambiguous information. Scenarios such as forensic investigations, incident-response coordination, and red-team/blue-team simulations illustrate these dynamic workflows. By systematically analyzing how LLMs are now being used to support such adaptive and interactive tasks, our study contributes a structured view of the current landscape and identifies the capabilities and challenges that define the next phase of LLM-driven cybersecurity automation.

2.2 Related surveys

Several recent surveys have examined the use of large language models within cybersecurity, reflecting a growing interest in how these models can support or automate a wide range of security-relevant activities. Some studies focus on a single domain, such as threat detection and threat intelligence extraction, and analyze how large language models can assist with identifying indicators, entities, and behavioural patterns in adversarial data [31]. Other surveys provide multi-domain coverage and describe applications spanning threat intelligence, vulnerability discovery, malware analysis, intrusion detection, and both defensive and offensive operations [4, 29, 30, 32, 34, 36]. A complementary line of work concentrates on evaluation practices, surveying benchmarks, assessment methodologies, and measurement challenges associated with applying large language models to cyber tasks [33]. More recent literature has begun examining autonomous cyberattack agents and tool-assisted attack chains, analyzing how LLMs may be used to orchestrate multi-step offensive activities [37]. Collectively, these studies offer extensive breadth and demonstrate the rapidly expanding set of cybersecurity applications enabled by large language models.

Despite this growing body of work, existing surveys tend to organize cyber tasks according to application domains or functional categories. In these surveys, the typical unit of analysis is a set of task labels, such as vulnerability identification, malware classification, intrusion detection, or secure code generation. This domain-oriented approach is valuable for summarizing where large language models have been deployed, but it provides limited insight into how different cyber tasks are structured, how they unfold over time, and what forms of reasoning or interaction they require from

Table 1 Positioning of this survey relative to prior surveys on LLMs in cybersecurity

Survey	Scope	Perspective	Unit of analysis	Contribution
Chen et al. [31]	Threat detection	Application	Detection task labels	Application survey
Ferrag et al. [32]	Multiple cyber domains	Application and capability	Domain labels and capabilities	Thematic survey
Gennari et al. [33]	LLM evaluation	Evaluation	Benchmarks, metrics, and methods	Evaluation framework
Hasanov et al. [34]	Cybersecurity landscape	Landscape	Domains and topical clusters	Systematic review
Hassanin and Moustafa [35]	Cyber defence	Defence	SOC functions and processes	Survey and challenge analysis
Motlagh et al. [29]	Offensive and defensive use cases	Application	Task categories and use cases	Thematic review
Tian et al. [36]	General cybersecurity applications	Application	Domain labels and examples	Systematic survey
Xu et al. [30]	Multiple cyber domains	Application	Domains and task types	Systematic review
Zhang et al. [4]	Cyber tasks and models	Application	Tasks, models, and datasets	Systematic review
Xu et al. [37]	Autonomous cyberattacks	Agent	Agents, tools, and attack stages	Agent-architecture analysis
This work	Cross-process cyber tasks	Task structure	Goals, grounding, collaboration, and complexity	Task characterization and operational mapping

models. As a result, prior surveys do not systematically examine how tasks differ with respect to their goals, the knowledge on which they depend, the allocation of responsibility between humans and models, or the reasoning demands they raise.

The present study offers a complementary perspective by focusing on the internal structure of cyber tasks rather than the domains in which they appear. We introduce a conceptual distinction between static and dynamic tasks and develop a framework that characterizes tasks in terms of their operational goals, knowledge-grounding requirements, collaboration modes, and levels of cognitive complexity. This task-structure perspective makes it possible to analyse cybersecurity activities as processes rather than labels, thereby offering a more granular view of how large language models must reason, adapt, and interact in order to perform them effectively. Furthermore, by mapping these structured task types to established operational frameworks such as the National Institute of Standards and Technology (NIST) Cybersecurity Framework (CSF)¹ and the MITRE

ATT&CK,² this work connects conceptual task structures to the practical workflows used in security operations and adversarial modelling. In doing so, the study provides a conceptual foundation for understanding how model capabilities relate to the procedural and dynamic nature of real-world cybersecurity activities, and it extends the scope of existing surveys by examining the implications of task structure for system design, evaluation, and deployment. Table 1 provides a structured comparison of existing surveys, which helps to clarify how the task-structure perspective adopted in this study represents a conceptually distinct orientation relative to prior application-based reviews.

3 Dynamic cyber automation tasks

To bring clarity to this broad category, we define and propose a characterization framework for dynamic cyber automation tasks based on four practical dimensions: operational goal, human–LLM collaboration mode, knowledge grounding, and cognitive complexity. This structure is intended

¹ <https://www.nist.gov/cyberframework>

² <https://attack.mitre.org/>

Table 2 Comparison of static and dynamic cyber tasks

Aspect	Static tasks	Dynamic tasks
Reasoning Steps	Primarily single-step operations that produce an immediate output based on direct input, with minimal dependency on evolving external context	Multi-step processes that require sequential decision-making, contextual adaptation, and integration of intermediate outcomes
Context Sensitivity	Minimal; task behavior may vary minimally depending on environment or input variation	High; task behavior varies based on system state, threat type, or user context
System Integration	No external system interaction beyond input/output	Often involves integration with other tools, LLM plays as one part of the system
Execution Logic	Typically treated as a single-turn task without externally visible branching or state tracking; reasoning steps are encapsulated internally	Requires explicit branching, iterative decision-making, and dependency on prior observations or actions during task execution
Human Role	Typically minimal or no involvement during execution, designed for one-shot automation	May involve human input or feedback in certain designs, but dynamic task structure—multi-step reasoning and adaptation—can also be executed autonomously
Examples	Phishing email classification, log anomaly detection, NER in threat reports	Incident investigation, remediation planning, adversary emulation
Representative Models	CyBERT, SecureBERT, FlowTransformer	GENTTP, RCACopilot, PentestGPT

to help researchers better understand, design, and evaluate dynamic tasks when working with LLMs in cybersecurity.

3.1 Definition

In the past few years, the demand for optimized cybersecurity solutions led to the development of new ideas in the field. This section presents works focused on dynamic cyber automation tasks, which indicate current trends and research gaps.

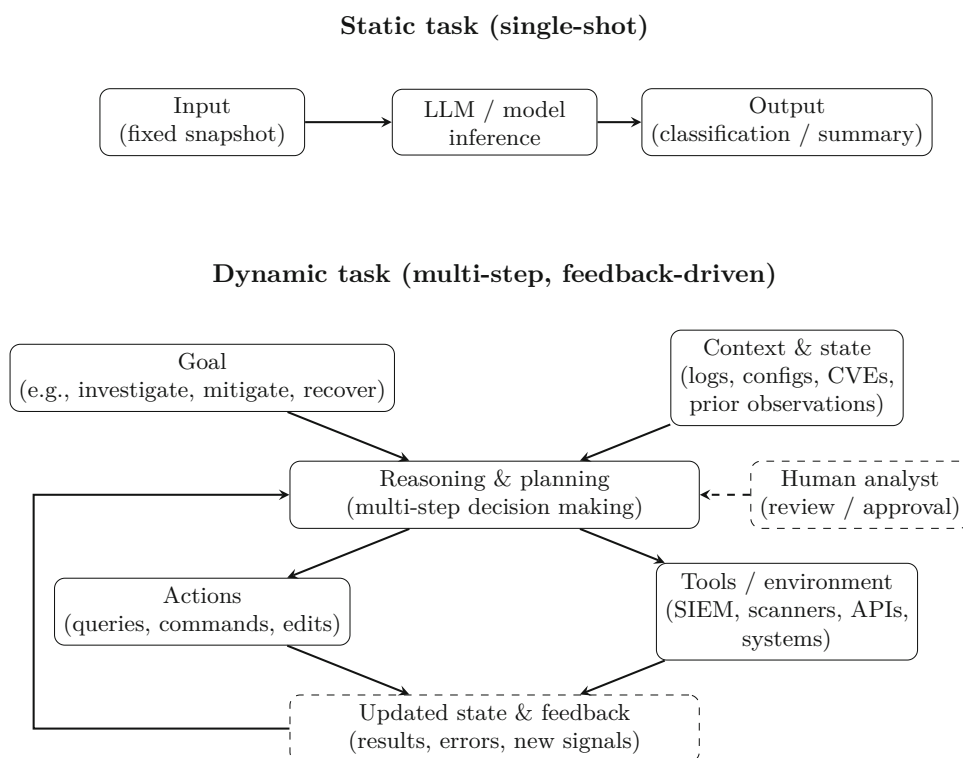
In the context of cybersecurity automation, tasks can broadly be divided into two categories: static and dynamic [28]. Static tasks typically involve single-step operations that produce deterministic outputs—such as classifying a log entry, extracting entities, or summarizing a report. These tasks are well-defined and do not require adaptation based on external context or changing inputs. Prior work has explored the use of LLMs for static tasks such as log anomaly detection, phishing classification, and named entity recognition from cyber threat intelligence reports. Models such as CyBERT [22], SecureBERT [23], and FlowTransformer [38] have shown effectiveness in these narrow, well-defined tasks.

Dynamic cyber tasks, by contrast, are multistep, context-aware processes that demand reasoning, decision-making, and adaptability. While the final output may sometimes be a fixed result—such as retrieving a flag in a CTF challenge—the path to reaching that output typically involves orchestrating a series of actions across multiple systems, interacting with the environment, and adapting to intermediate observations. Unlike static tasks, dynamic operations frequently require conditional logic, iterative planning, and tool use to successfully accomplish the security objective.

Inspired by the framework proposed by Sultana et al. [28], we characterize dynamic tasks by the following properties also presented in Table 2 as a comparison of static and dynamic cyber tasks:

- *Reasoning Steps*: Dynamic tasks require explicit multi-step reasoning where each step may depend on the outcome of previous actions or observations. In static tasks, reasoning is typically encapsulated within a single inference, without externally observable sequential decision-making.
- *Context Sensitivity*: Dynamic tasks require the model to adjust its behavior based on evolving system state, new evidence, and changing operational constraints. Unlike static tasks, where outputs are determined solely by the immediate input, dynamic tasks depend on how prior observations, threat conditions, or user intent modify the reasoning process. This adaptability is essential for tasks such as incident investigation or threat analysis, where conclusions must evolve as new information appears.
- *System Integration*: Dynamic tasks often require interaction with external tools, platforms, or data sources, making the LLM one component in a larger operational loop. Instead of producing a single static output, the model may need to issue commands, interpret tool responses, and revise its plan based on system feedback. This integration reflects real cybersecurity workflows, where effective automation depends on coordinating actions across multiple systems rather than performing isolated text-based inference.
- *Execution Logic*: Dynamic tasks typically involve explicit branching, iterative decision-making, or state tracking

Fig. 1 Typical workflow for static and dynamic cyber tasks. Static tasks are one-shot mappings from a fixed input snapshot to a single output. Dynamic tasks are multi-step, feedback-driven processes that integrate goals, evolving context, external tools, and updated state across iterations



that is observable during task execution. In contrast, static tasks are often designed as single-turn processes where any internal reasoning remains encapsulated and no external adaptation or workflow branching is required.

- **Human Role:** Many dynamic tasks benefit from or require human feedback, review, or approval. However, human participation is common but not required for a task to be classified as dynamic. The defining property is multi-step, context-aware, adaptive reasoning.

Examples of dynamic tasks include: (i) Investigating an alert by gathering additional logs, enriching data, and recommending next steps; (ii) Generating and executing a remediation playbook based on incident severity; (iii) Writing firewall rules or IAM policies based on a natural language description of a threat or requirement; and (iv) Simulating adversarial actions in a controlled environment to test detection coverage.

While Table 2 summarizes typical characteristics of static and dynamic cyber tasks, it is important to note that the classification often depends on how the task is designed and executed. Certain tasks, which are traditionally viewed as static—such as classification, summarization, or mathematical problem-solving—can be framed in a dynamic way if they require iterative interactions, external tool usage, or environment feedback. In this view, dynamic tasks are those that involve agentic workflows, where the system reasons over multiple steps, interacts with external tools or environments, and adapts its plan based on intermediate results. Our frame-

work emphasizes these operational characteristics, and we acknowledge that the static or dynamic nature of a task is not absolute but context-dependent. Figure 1 illustrates the typical workflow for static tasks and dynamic tasks.

3.2 Characterization framework for dynamic cyber tasks

Figure 2 presents the characterization of dynamic cyber tasks. This section discusses its specific details, including operational goal, human–LLM collaboration mode, knowledge grounding, and cognitive complexity. A characterization framework is necessary because existing cybersecurity surveys and application studies typically organize LLM-related work by domain labels, such as threat intelligence, vulnerability detection, or incident response, without examining how the underlying tasks differ in structure, execution, or cognitive demands. As discussed in Sect. 2, this domain-oriented view is useful for summarizing where LLMs have been applied, but it provides little insight into how these tasks unfold, what forms of reasoning they require, or what external knowledge they require, and how responsibility is distributed between LLMs and human analysts. Dynamic cyber tasks, in particular, vary widely in their goals, feedback dependencies, knowledge grounding requirements, collaboration modes, and reasoning demands. Without a structured way to characterize these differences, it becomes difficult to compare systems, identify capability gaps, or design evaluations

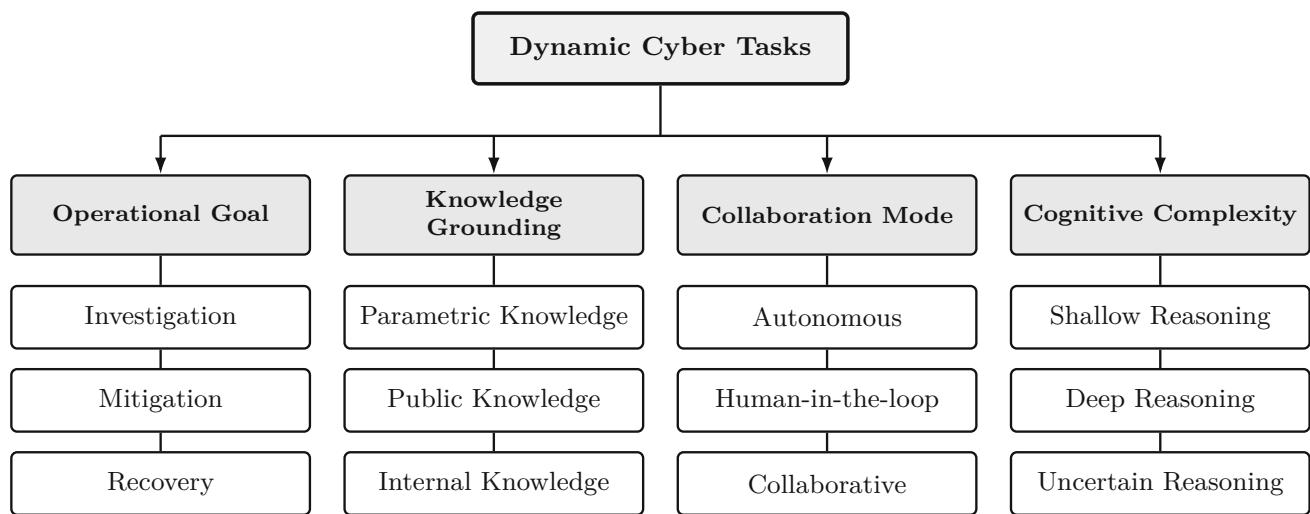


Fig. 2 Characterization of dynamic cyber tasks

that reflect the multi-step, context-dependent nature of real-world cyber operations. The characterization introduced here addresses this gap by offering a task-structure-oriented lens that captures the operational goals, knowledge-grounding requirements, collaboration modes, and cognitive complexity underlying dynamic cyber workflows. This framework provides the conceptual foundation needed to analyze emerging LLM-enabled systems consistently and to reason about the capabilities required for trustworthy automation in complex cybersecurity environments.

3.2.1 Operational goal

Dynamic cyber tasks can be grouped based on their high-level objectives. We adopt the structure of incident response workflows as described in operational standards such as the NIST Cybersecurity Framework (CSF) [39]. Based on this structure, we group task goals into three categories: *investigation*, *mitigation*, and *recovery*. Investigation-oriented tasks focus on understanding what happened, such as tracing an attack through logs or identifying the initial point of compromise. Mitigation tasks are action-driven and involve responding to an active threat, for example, blocking a malicious IP, deploying a firewall rule, or isolating a compromised host. Recovery tasks come after threat containment and focus on restoring systems, conducting post-incident analyses, and generating incident reports. While these categories originate from traditional incident handling processes, we adapt them here to frame and classify the diverse goals of dynamic cyber task automation.

3.2.2 Knowledge grounding

Another key aspect of dynamic tasks is the type of knowledge required for the LLM to reason reliably. Some tasks

can be completed using the model's *parametric knowledge* together with the evidence directly supplied as part of the task, such as summarizing a provided incident report or explaining a well-established vulnerability. Other tasks require *public knowledge*, including current CVE records, vendor advisories, threat-intelligence feeds, and MITRE ATT&CK information. More operationally dependent tasks require *internal knowledge*, such as local logs, asset inventories, software bills of materials, network configurations, internal policies, and incident-response playbooks. These categories are not necessarily mutually exclusive, as a task may combine public threat information with local operational context. The required grounding therefore determines whether an LLM can be used directly or must be supported by retrieval, organizational data, and appropriate access controls.

3.2.3 Collaboration mode

Dynamic tasks also vary in the degree of autonomy that the LLM is expected to have. In some settings, the model is given a goal and left to carry it out independently, planning the steps and adapting as needed; these are what we refer to as *autonomous* tasks. In other cases, the model works alongside a human analyst, offering suggestions or taking actions based on prompts, which we would call *human-in-the-loop* setups. A third category involves *collaborative* tasks, in which multiple LLM agents, automated components, or human analysts coordinate throughout the workflow. Human-in-the-loop designs introduce explicit review or approval points, whereas collaborative designs involve sustained coordination among participants across multiple stages of the task. These different execution styles influence not only task design but also how performance should be measured and evaluated.

3.2.4 Cognitive complexity

Finally, dynamic tasks differ in how much reasoning they require. We divide tasks into three levels: *shallow reasoning*, *deep reasoning*, and *uncertain reasoning*. Shallow tasks are relatively straightforward and can be completed in a few steps, such as identifying a phishing domain and blocking it. Deep reasoning tasks involve deeper decision-making, where the model must follow leads, prioritize actions, or navigate uncertainty. Uncertain reasoning is more complex; the model may need to infer missing information, ask clarifying questions, or even revise its plan as new data come in. These are the kinds of tasks where the limitations and potential of LLMs become most apparent. Measuring how well models handle these situations is a key part of evaluating their real-world utility.

Therefore, the complexity of dynamic cyber automation tasks demands advanced solutions to effectively improve cybersecurity aspects. In the next section, we explore the applications of LLMs in this context.

3.3 Mapping the characterization framework to NIST and MITRE

Tables 3 and 4 contrast representative cyber tasks across defensive, offensive, and cross-domain frameworks, showing how each conventional static task can evolve into a dynamic process when coupled with feedback-aware or continuously adaptive reasoning. Each row pairs a static case, where an LLM performs the task once on fixed inputs without external context, with its dynamic counterpart, where decisions or outputs are iteratively refined through environmental signals, streaming data, or verification feedback. The NIST section illustrates defensive task automation, such as risk assessment, secure code generation, and adaptive incident response, whereas the MITRE ATT&CK portion captures feedback-driven offensive behaviors like polymorphic malware generation and adaptive credential harvesting. The cross-domain section presents supporting capabilities such as adversarial testing, zero-trust integration, and autonomous threat hunting, which are not in those frameworks but are also important tasks in cybersecurity research.

These mappings do not imply that every cybersecurity task should be delegated to an LLM or automated end to end. LLM support is most suitable when a task requires the interpretation of heterogeneous or unstructured evidence, cross-source synthesis, contextual reasoning, or adaptive planning. High-impact actions, including containment, access-control changes, remediation, and exploit execution, should generally remain human-gated or constrained by deterministic safeguards. Structured and repeatable tasks with stable rules and bounded action spaces may remain better suited to conventional automation.

Within the NIST defensive framework, this transformation begins at the Identify and Protect stages. Threat-intelligence pipelines that once produced static reports now process heterogeneous logs and continuous CTI feeds; each new indicator of compromise becomes a feedback event prompting real-time correlation and graph updates. Vulnerability detection, formerly a static scan of source or firmware, turns adaptive when embedded into continuous-integration pipelines that trigger re-analysis after each commit or when new CVEs appear. Even policy and compliance generation can behave dynamically when regulatory changes or configuration drifts are monitored automatically and compliance documents are regenerated accordingly. Risk assessment systems similarly evolve from one-time evaluations into continuous exposure-tracking processes that recompute risk scores as assets and threat surfaces change. In the Protect phase, secure-code generation and patch repair incorporate immediate verification signals, the model can rewrite insecure functions or refine a patch based on test or bounded-model-checking feedback, creating a self-healing loop. Firewalls and network defenses become reinforcement-learning controllers that tune filtering rules from telemetry statistics, while access-control and privacy mechanisms employ zero-trust scoring to adapt authorization decisions per session. Even governance and bias auditing can update dynamically when fairness metrics or model-drift indicators feed directly into compliance dashboards.

At the Detect, Respond, and Recover stages, the degree of dynamism intensifies. Intrusion and anomaly detection rely on streaming telemetry and sliding-window baselines that update continuously as behavioral distributions drift, while malware classifiers incorporate runtime traces from sandboxes to retrain behavioral embeddings. Phishing detection adapts its linguistic filters by fine-tuning on newly captured campaigns, maintaining sensitivity to evolving deception styles. In incident response, alerts from detection modules trigger LLM reasoning that generates or revises containment playbooks; analyst confirmation or failure becomes feedback for future actions, forming a human-in-the-loop Security Orchestration, Automation, and Response (SOAR) cycle. Cyber-deception systems and honeypots maintain conversational or behavioral state with adversaries, using each interaction to refine response tone and lure credibility. Reinforcement-learning-based defense agents close the loop entirely by sensing environmental conditions and adjusting configurations to maximize long-term resilience. Even documentation tasks such as explainable security reporting, can regenerate narratives automatically whenever model weights or alert contexts change, while DDoS detection and mitigation engines continuously adjust thresholds through reinforcement or heuristic control using live network feedback.

Table 3 Static versus dynamic characterization of LLM-driven cyber tasks by framework domain in defensive aspect

Framework	Functional area	Task category	Dynamic case	Static case
NIST Defensive	Identify	Threat Detection & Intelligence	Parse heterogeneous logs with streaming ingestion and incremental IoC correlation	One-time log analysis on archived data
			Extract and correlate CTI from live reports and feeds via online entity linking and rolling updates	Manual CTI extraction from fixed reports
		Vulnerability Detection (Code)	Continuous code and firmware reanalysis triggered by new commits or CVEs/Event-driven LLM scanning	One-shot LLM code scan with frozen prompt; no feedback or retraining
		Policy & Compliance Generation	Policy-as-code regeneration when regulations or configurations evolve	Single compliance report from current regulations
	Protect	Risk Assessment	Continuous risk reassessment as LLMs fuse live asset inventories and vulnerability updates	One-shot risk summary from static CVE/SBOM lists
		Secure Code Generation and Prevention	Real-time secure code generation in IDEs with interactive, state-aware prevention	Generate secure templates without runtime context
		Vulnerability Repair/Patch Generation	Iterative patch synthesis refined through test feedback, forming a self-healing loop	Suggest static patch; no iterative test feedback
		Firewall/Network Defense Automation	Continuous rule generation and tuning from live telemetry	Rule set generation from static policy description
	Detect	Access Control & Privacy Enforcement	Context-aware anonymization and zero-trust scoring based on live context	Apply predefined redaction templates
		Intrusion/Anomaly Detection	Online host or network anomaly detection with sliding windows and adaptive baselines	Evaluate model on historical dataset only
		Malware Detection & Classification	Continuous re-learning from sandbox logs and unseen behavioral patterns	Classify samples once using frozen model
			Real-time alert generation with live sequence summarization	Trigger fixed-threshold alerts
	Respond	Phishing Detection	Continual adaptation to linguistic drift during classification	Classification of email batch without drift adaptation
		Incident Response & Mitigation	Event-triggered containment and SOAR playbook execution	Post-incident report with no live update
	Recover	Cyber Deception/Honeypots	Adaptive attacker engagement through conversational state and dynamic responses	Pre-scripted honeypot interactions
		Autonomous/Adaptive Defense Agents	Self-healing configuration updates via reinforcement feedback loops	Manual recovery or fixed playbooks
		Explainable Security/Reporting	Generate evolving justifications based on analyst feedback	Template-based alert explanation
		DDoS Detection & Mitigation	Real-time detection and mitigation via stream features and RL-based threshold control	Fixed thresholds or reasoning prompt

A similar pattern appears in the MITRE ATT&CK offensive framework, where adaptivity is often the attacker's competitive advantage. Reconnaissance evolves from static scraping into recursive exploration as LLM-driven crawlers follow newly discovered entities and update OSINT graphs. Exploit and payload development adopts an iterative fuzzing cycle in which generated exploits are tested, failures analyzed, and parameters automatically modified. Phishing and social-engineering campaigns refine tone and timing through response feedback such as open rates or sentiment analysis. Malware-generation systems mutate payloads according to

antivirus or EDR responses, effectively performing evolutionary optimization, while privilege-escalation and persistence scripts probe the environment and regenerate sequences when earlier methods fail. Evasion and obfuscation tools execute gradient-guided mutation until detection confidence falls below a threshold, and password-guessing attacks adjust candidate probabilities using timing or failure feedback. Data-exfiltration and command-and-control infrastructures adapt channel encoding and beacon intervals in reaction to network blocks, and even ransomware can vary encryption routines based on defensive counter-signals. In all these

Table 4 Static versus dynamic characterization of LLM-driven cyber tasks by framework domain in attack and cross-domain aspects

Framework	Functional area	Task category	Dynamic case	Static case
MITRE ATT&CK Offensive	Reconnaissance/ Resource Development	Automated Reconnaissance	Continuous OSINT monitoring with adaptive search queries reacting to target changes	Generate target profile from OSINT snapshot
		Exploit/Payload Development	Exploit code refinement based on runtime or detection feedback	Generate exploit or PoC from given CVE description
	Initial Access	Phishing/Social Engineering	Persona-mimicking spear-phishing with iterative personalization and response-driven edits	Compose single phishing message without iterative refinement
	Execution	Malware Generation	Payload mutation guided by AV/EDR feedback for evasion	Produce payload without testing and mutation
	Persistence/ Privilege Escalation	Exploit Chain Assembly	Conditional escalation steps updated through environment sensing and replanning	Generate fixed sequence of escalation steps
	Defense Evasion	Evasion & Obfuscation	Polymorphic mutation via test–mutate loops against detectors	Generate obfuscated sample without iteration
	Collection/ Exfiltration	Data Harvest & Exfiltration	Covert channel tuning based on real-time blocking feedback	Generate exfiltration script without further tuning
	Command & Control	Remote Access/Proxy Control	Dynamic beacon management with timing/channel adaptation vs. blocking	Propose beacon schedule without timing adaptation
	Impact	Ransomware/ Destructive Scripts	Adaptive ransom note tuning based on victim behavior or system feedback	Generate encryption or ransom script without adaptation
	Cross-Domain/ Supporting	Adversarial AI & Robustness	Adversarial Example Generation	Online adversarial testing with adaptive modification of attack inputs based on detector responses
Zero-Trust Integration			Continuous re-authentication and adaptive risk scoring based on context	Generate static access policy from fixed rules
Autonomous Threat Hunting			Proactive IoC exploration via agent-based feedback loops	Generate IoC correlation report, no iteration

cases, feedback—whether from system responses, network latency, or the model itself—creates an adaptive adversarial loop.

Cross-domain and supporting tasks further generalize the feedback principle. Adversarial-example generation and robustness evaluation form red-team/blue-team feedback frameworks in which perturbed inputs are repeatedly tested against detectors and refined until misclassification occurs, prompting defensive retraining. Federated-learning defenses exhibit inherent iteration as local clients update and aggregate global models asynchronously to handle data drift. Zero-trust architectures continuously reevaluate contextual signals—device posture, geolocation, and user behavior—to recalculate trust on each transaction, while autonomous threat-hunting agents traverse event graphs, expanding search boundaries when anomalies are discovered. Even supporting domains such as explainability and education become adaptive. Explanation systems regenerate reports as model

decisions evolve, and tutoring environments adjust difficulty and scenario content in response to learner performance metrics.

In conclusion, these processes demonstrate that dynamism in cybersecurity is no longer tied to the task category but to the feedback structure surrounding it. LLMs supply the interpretive reasoning needed to transform unstructured signals, such as telemetry, textual reports, user behavior, or human feedback, into actionable adjustments of system state. What was once a binary distinction between static and dynamic tasks now forms a continuum from snapshot reasoning to continuous autonomous adaptation. Static analysis tools become dynamic when embedded in event-driven pipelines; response systems become semi-autonomous through reinforcement learning; and even governance or education functions gain adaptivity when linked to real-time monitoring. This reconceptualization implies that the next generation of cyber-automation research should focus less on isolated model

performance and more on designing secure, interpretable feedback loops that govern when and how LLMs update their reasoning in live environments.

4 Dynamic cyber automation tasks with LLMs

While Sect. 3.3 mapped dynamic cybersecurity processes along functional layers of the NIST and MITRE frameworks, this section examines how those dynamics materialize within operational domains where LLMs have shown measurable progress. The seven domains selected, including threat intelligence, data privacy and security, vulnerability detection, incident response, malware detection, intrusion detection, and red team automation, represent the most frequently investigated areas in recent LLM for cybersecurity literature [6, 29, 30]. They collectively span both defensive (NIST Identify and Recover) and offensive (MITRE ATT&CK) phases, providing balanced coverage of proactive, reactive, and adversarial tasks. Each domain therefore serves as an application layer through which the dynamic properties introduced in Sect. 3 can be concretely observed and evaluated. The systems discussed in this section are representative rather than exhaustive. We focus on studies that exhibit explicit characteristics of dynamic automation, including multi-step reasoning, tool interaction, environmental feedback, state maintenance, adaptive planning, or sustained human-LLM collaboration. Studies limited to one-shot prompting, static classification, or isolated text generation are not treated as representative dynamic systems.

Before discussing the individual domains, Fig. 3 summarizes the common workflow transformation introduced by LLM augmentation. In the conventional workflow, analysts manually collect and correlate evidence, interpret tool outputs, make operational decisions, and design or revise action plans. In the augmented workflow, the LLM partially automates these repeated activities by retrieving and synthesizing context, making intermediate decisions, coordinating existing security tools, and continuously updating its reasoning from live outputs. Human analysts remain involved through guidance, constraint setting, review, correction, and escalation handling.

The extent of this workflow transformation varies across tasks. Some systems remain limited to retrieval and decision support, whereas others coordinate tools, make intermediate operational decisions, and revise actions through environmental feedback. The appropriate degree of automation also depends on the task goal, required knowledge grounding, collaboration mode, and cognitive complexity. Table 5 characterizes the representative systems along these dimensions, while Sects. 4.1–4.7 discuss their domain-specific implementations, evidence of dynamism, and limitations.

4.1 Threat intelligence

Threat intelligence automation has traditionally relied on static extraction and correlation processes. Conventional models identify indicators of compromise from reports, classify entities, and populate structured feeds for analysts to review. Each dataset is processed independently, producing fixed outputs that do not evolve as new information appears. This static paradigm corresponds to single-step reasoning in Table 2, where models function primarily as text classifiers without contextual awareness or iterative refinement. In practice, however, CTI is highly temporal and context-dependent. Adversarial campaigns evolve, tactics and procedures mutate across sources, and network telemetry continuously generates new signals. Static models are therefore unable to capture the adaptive and evolving nature of real-world threat environments.

Recent studies have begun reframing CTI as a dynamic reasoning problem in which LLMs act as interpreters that continuously revise their understanding of ongoing threats. Dynamism in this context arises from feedback and contextual adaptation rather than one-pass extraction. Three main mechanisms are involved. The first is temporal feedback, in which new threat data or logs cause the system to revisit earlier correlations and update prior conclusions. The second is retrieval feedback, where the model queries external repositories such as MISP or the MITRE ATT&CK framework and integrates the retrieved information into its reasoning process. The third involves analyst or environmental feedback, where human verification or system alerts guide the next round of analysis. These mechanisms correspond to an investigation-oriented goal, public and internal knowledge grounding, autonomous or human-in-the-loop collaboration, and deep reasoning within the framework introduced in Sect. 3.2. In this setting, CTI becomes a continuously adaptive process in which reasoning depth and environmental awareness are central to analytic quality.

Several systems illustrate this transformation. Fayyazi et al. [40] compared encoder-based and decoder-based LLMs for mapping natural-language attack descriptions to MITRE ATT&CK tactics. Their retrieval-augmented design allows contextual reasoning that improves as additional procedural evidence becomes available. The ThreatWise AI framework [41] operationalizes dynamic CTI at a larger scale by aggregating and enriching indicators from multiple sources including honeypots, Wazuh logs, social-media posts, and CERT advisories. The system enriches incoming data through entity recognition, correlation, and prioritization before sharing updated intelligence through MISP. Its continuous ingestion and enrichment pipeline exemplifies the adaptive feedback loop described in Sect. 3.3, where new signals not only extend the dataset but also modify the analytic model itself. These examples demonstrate how dynamic reasoning transforms

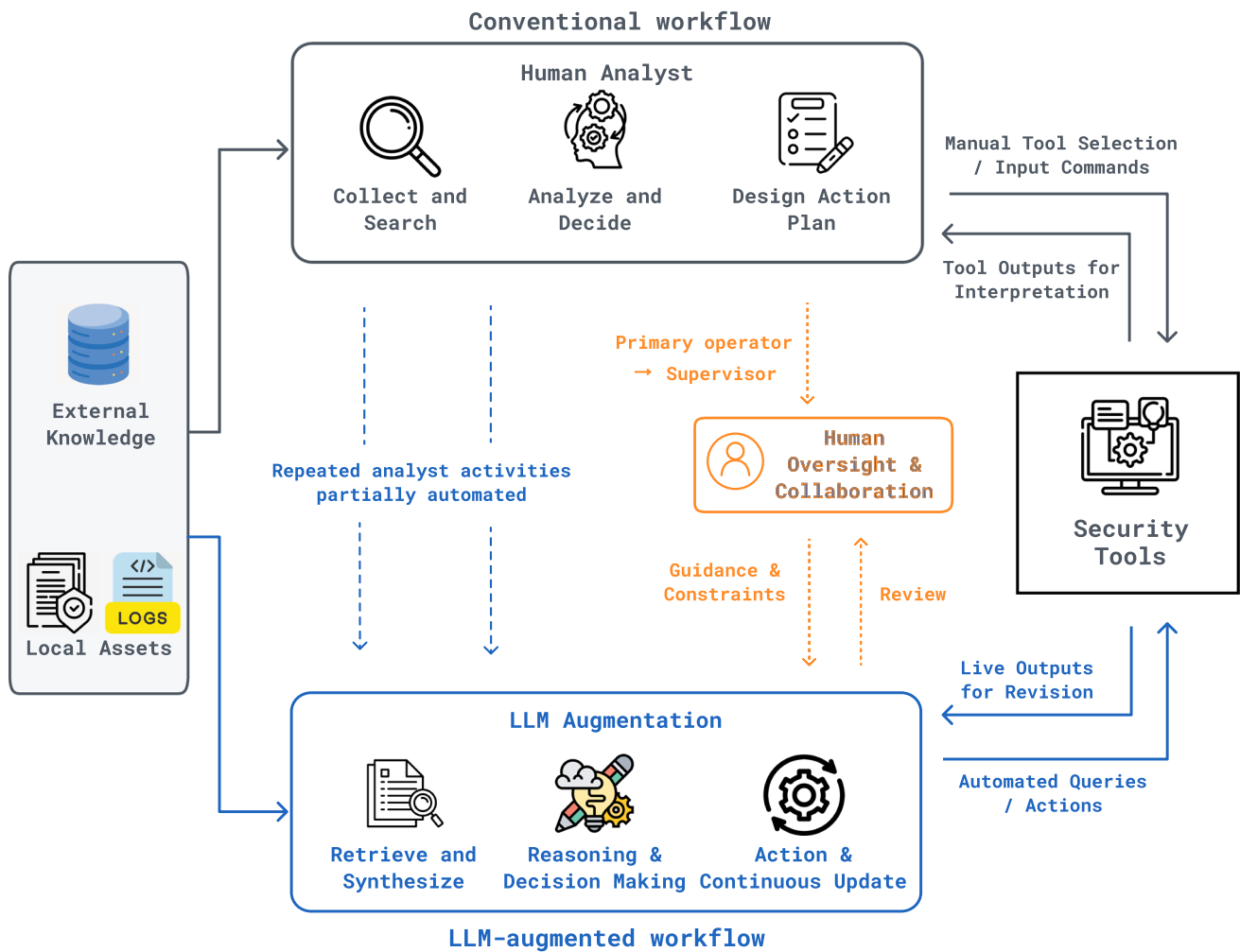


Fig. 3 Comparison of conventional and LLM-augmented cybersecurity workflows. In the conventional workflow, analysts collect and correlate evidence, interpret tool outputs, make operational decisions, and revise action plans. LLM augmentation partially automates these activities through contextual retrieval, evidence synthesis, intermediate decision-

making, automated tool coordination, and continuous revision from live outputs. Human involvement remains through guidance, constraint setting, review, correction, and escalation handling. The figure represents a shared workflow pattern rather than a fixed implementation for every cybersecurity domain

CTI from static parsing into an evolving decision-support process that fuses multiple data streams and adjusts interpretations as the threat landscape changes.

4.2 Data privacy & security

The automation of data privacy and security management has been seen as a static process. Traditional models identify sensitive entities, enforce predefined access-control rules, or apply encryption and anonymization based on fixed policies. Once a decision is made, no further contextual reasoning occurs, and new data, user behavior, or environmental changes rarely influence subsequent outcomes. Such systems correspond to simple classification or transformation tasks that lack environmental feedback and contextual mem-

ory. However, modern privacy protection must continuously adapt to evolving contexts, such as regulatory changes, dynamic user consent, and real-time data usage patterns. The complexity of these evolving factors positions privacy management as a natural candidate for dynamic cyber-automation, where decisions must be reviewed and revised as conditions change.

Recent work in privacy automation has begun shifting from fixed masking and static access-control rules toward dynamic, feedback-driven protection mechanisms. Systems such as PrivacyAsst [42] illustrate how privacy management behaves as a dynamic cyber task: decisions evolve over time, react to changing inputs, and adapt based on tool interactions and contextual feedback rather than being applied once in a single pass. In PrivacyAsst, the model does not simply

Table 5 Characterization of representative LLM-enabled cyber tasks. HITL denotes human-in-the-loop

Domain	Representative task or system	Task profile	Operational profile
Threat Intelligence	TTP extraction from unstructured reports [40]	Goal: Investigation Reasoning: Deep	Grounding: Public Knowledge Collaboration: Autonomous
	Cross-source CTI correlation and enrichment [41]	Goal: Investigation Reasoning: Deep, uncertain	Grounding: Public/Internal Knowledge Collaboration: Autonomous
Data Privacy & Security	Privacy-preserving tool orchestration [42]	Goal: Mitigation Reasoning: Deep, uncertain	Grounding: Internal Knowledge Collaboration: Autonomous
Vulnerability Detection	Context-aware CVE triage [43]	Goal: Mitigation Reasoning: Deep	Grounding: Public Knowledge Collaboration: Autonomous
	Post-exploitation remediation planning [44]	Goal: Recovery Reasoning: Deep	Grounding: Public/Internal Knowledge Collaboration: Autonomous/Collaborative
Malware Detection	TTP extraction and behavior modeling from interpreted malware [26]	Goal: Investigation, mitigation Reasoning: Deep, uncertain	Grounding: Parametric/Public Knowledge Collaboration: Autonomous
Intrusion Detection	Anomaly behavior-chain tracing [45, 46]	Goal: Investigation Reasoning: Deep	Grounding: Internal Knowledge Collaboration: Autonomous
Incident Response	Root-cause analysis from multi-source incident data [47, 48]	Goal: Investigation Reasoning: Deep	Grounding: Internal Knowledge Collaboration: Collaborative/HITL
Red Teaming Automation	Feedback-driven multi-step penetration testing [25]	Goal: Mitigation Reasoning: Deep	Grounding: Parametric/Internal Knowledge Collaboration: HITL

anonymize or redact data through predetermined templates. Instead, it continuously evaluates user inputs, tool responses, and task context to determine how privacy-preserving transformations should change across reasoning steps. When new information arrives—such as a change in task type, user intent, or intermediate tool output—the agent reassesses its decisions, adjusts its strategy, and reconfigures the privacy pipeline accordingly. This creates a closed feedback loop, where privacy protection is repeatedly refined based on environmental signals rather than fixed policy execution. This dynamic behavior also reflects deeper reasoning. The agent must infer which parts of the data are sensitive under the current task goal, determine how different transformations will affect downstream tool performance, and revise its approach when transformed outputs fail to satisfy correctness or privacy requirements. Rather than producing a one-time anonymization, PrivacyAsst performs iterative, context-aware privacy reasoning, updating its strategy in real time as the task evolves. Within our framework, the system exemplifies the transition from static privacy operations to adaptive mitigation tasks with multimodal inputs, continuous feedback, and uncertain reasoning. The privacy guarantees are maintained not through rigid masking rules but through ongoing adjustment, where each step influences the next. This shift highlights how privacy protection itself becomes a dynamic cyber-automation process—one that requires tool

interaction, contextual interpretation, and iterative refinement to remain effective under changing conditions.

4.3 Vulnerability detection

Traditional vulnerability detection frameworks are inherently static. They rely on fixed signatures, rule-based scanning, or syntactic matching to identify flaws within source code or compiled binaries. Once the scan is completed, no further reasoning occurs, and the system does not adapt when codebases evolve or new vulnerability definitions emerge. Such workflows correspond to the identification stage of the NIST framework and to the single-step reasoning level of Table 2, where vulnerability discovery is treated as an isolated task rather than a continuous analytical process. However, real-world software ecosystems change rapidly, with new code commits, dependency updates, and CVE disclosures occurring daily. Static detection therefore struggles to maintain accuracy and timeliness, as each analysis represents only a snapshot in time. The growing complexity of modern software supply chains, particularly in automotive and IoT contexts, demands a continuous vulnerability reassessment that evolves in step with system updates.

A recent advance in this direction is the framework proposed by Wu et al. [43], which introduces an LLM-driven approach for identifying vulnerable functions from

natural-language CVE descriptions. Instead of relying solely on handcrafted features or syntactic similarity, the system performs in-context learning and semantic retrieval to extract key vulnerability entities—such as attack vectors, root causes, and affected components—and align them with candidate functions in source code. The model iteratively interprets CVE text, retrieves semantically relevant code fragments, and applies reasoning to infer which functions are most likely vulnerable. This approach enhances precision and recall by incorporating contextual semantics and natural-language inference, demonstrating how LLMs can reason over heterogeneous data modalities rather than applying rigid pattern rules. Within the characterization framework established in Sect. 3, this method exemplifies deep reasoning and context-aware investigation, achieving partial dynamism through adaptive inference and contextual alignment.

Nevertheless, the framework operates as a batch-level pipeline and does not maintain long-term feedback or a continuous state. Once the reasoning process completes for a given CVE, the system does not re-evaluate results when new commits or updated vulnerability information become available. Its adaptivity occurs within a single analytic cycle rather than across time, limiting autonomy and feedback depth. In operational environments, however, such a system could serve as the reasoning core of a dynamic pipeline in which new CVE releases or code changes automatically trigger re-analysis. Embedding this LLM-based reasoning into CI/CD pipelines, software bill-of-materials (SBOM) monitoring, or automated patch verification would realize the fully dynamic loop mentioned in the previous section, where vulnerability assessment evolves continuously alongside the software life-cycle.

Beyond static analysis, recent work has expanded vulnerability detection into multi-stage, interaction-driven workflows. A representative example is PenHeal [44], a two-phase framework that performs LLM-guided penetration testing followed by optimized remediation planning. Rather than treating vulnerability detection as a standalone classification task, PenHeal integrates reconnaissance, scanning, exploitation attempts, and iterative feedback from external tools to validate the presence and operational impact of security weaknesses. This design makes vulnerability assessment inherently dynamic because the system continuously updates its internal attack plan based on command outputs, revises hypotheses when exploits succeed or fail, and adapts subsequent actions accordingly. Such feedback-driven refinement is essential in realistic environments, where vulnerability manifestations depend on configuration state, running services, privilege boundaries, and tool responses—conditions that cannot be captured through static code or signature matching alone. Moreover, PenHeal extends detection into the recovery phase by generating cost-aware remediation

strategies, demonstrating why dynamic workflows are critical for bridging the gap between identifying a flaw and restoring system integrity.

4.4 Malware detection

Malware detection has been an exercise in static pattern recognition. Signature-based engines and feature-driven classifiers dominate production environments, relying on pre-extracted byte sequences, opcode n-grams, or fixed behavioral templates. Such methods succeed when threats resemble known families but falter as malware evolves through obfuscation, packing, or polymorphism. The fundamental limitation lies in their static perspective. Each binary is assessed as a closed object, detached from context, history, or intent. As adversaries increasingly craft malware that changes form while preserving function, the task of detection begins to demand reasoning about how code behaves rather than what it looks like [49]. Static detection, by its very nature, cannot capture these evolving semantics or learn from newly observed behaviors, revealing the need for systems that interpret malicious activity as an unfolding process rather than a fixed artifact.

Recent LLM-enabled frameworks illustrate how malware detection can evolve into a dynamic reasoning process. Instead of scanning binaries against fixed rules, these models interpret program behavior through multi-step inference, linking textual descriptions, metadata, and execution traces to higher-level attack semantics. One example system is GENTTP [26]. By decomposing analysis into reasoning sub-tasks such as identifying deception strategies, reconstructing execution steps, and mapping them to ATT&CK tactics, the model creates an internal feedback loop in which each inference refines the next. This decomposition, together with structured prompting and chain-of-thought reasoning, allows contextual adaptation to diverse code structures and unseen malware behaviors. Across such frameworks, feedback integration occurs when generated behavioral hypotheses are re-evaluated against new evidence. Contextual adaptation emerges as the model adjusts reasoning pathways to reflect changing code semantics. Multi-step reasoning allows causal reconstruction of attack sequences rather than one-shot classification. Operating in a zero-shot setting, these agents also exhibit a degree of autonomy, conducting end-to-end interpretation without labeled data or predefined signatures. As their analyses accumulate, the resulting knowledge bases expand over time, forming a persistent memory that enhances future inference. Collectively, these properties mark the transition from static evaluation to continuous, context-aware reasoning, where each detection refines the system's understanding of adversarial behavior.

4.5 Intrusion detection

Traditional intrusion detection systems (IDS) rely on static signatures or pre-trained classifiers that identify anomalies as fixed deviations from a baseline. Once deployed, these models operate passively, issuing alerts without considering temporal evolution or contextual meaning. While effective for known threats, they struggle to recognize emerging behaviors or adapt to shifting workloads and attacker tactics. As network behavior and adversarial tactics evolve continuously, intrusion detection must move beyond static pattern recognition toward reasoning systems that interpret events as part of a changing operational context.

Recent work has demonstrated how LLMs can operationalize this transition through dynamic task behavior. Darvishi et al. [45] introduce an adaptive tracing framework that employs a language model to interpret kernel-level trace sequences and regulate what to monitor based on real-time feedback. Rather than recording every event, the model analyzes both event order and duration through dual reasoning modules and autonomously decides when to activate or suspend detailed tracing. This selective monitoring forms an internal feedback loop that optimizes performance while maintaining visibility. The approach demonstrates feedback integration, as tracing scope changes dynamically with evolving system conditions, and contextual adaptation, since reasoning about timing and frequency reflects changing workload patterns. Its multi-objective inference across sequence and duration tasks is a sign of multi-step reasoning, while autonomous decisions on tracing boundaries show a degree of self-directed operation. By learning new behavioral norms over time, the model also exhibits the foundation of persistent memory, qualifying it as a dynamic cyber-automation process.

At the user level, Song et al. [46] propose the Insider Threat Detection Language Model (ITDLM), which extends dynamism to human-centered telemetry. ITDLM converts user activity logs into language representations and applies a two-stage fine-tuning process. The first stage learns general behavioral patterns, and the second introduces user-specific adaptation based on temporal and contextual feedback. This structure creates a self-refining reasoning loop where each stage improves the next, establishing feedback integration between general and personalized models. As the system learns to interpret evolving work rhythms and interaction patterns, it demonstrates contextual adaptation to the environment. The detection pipeline further embodies multi-step reasoning through sequential stages of topic extraction, behavioral contrast, and causal inference. Its ability to operate without labeled data and autonomously update its embeddings indicates autonomy, while the accumulation of user-specific knowledge across sessions reflects persistent memory. These dynamic dimensions enable ITDLM to detect

insider threats not by matching predefined signatures but by reasoning over evolving behavioral semantics.

4.6 Incident response

The typical incident response process is rule-driven and static. Most systems follow fixed playbooks or deterministic workflows that map alert types to predefined containment or recovery actions. Once executed, these workflows terminate without considering feedback from evolving incident data, secondary alerts, or analyst input. This rigid design corresponds to the response phase of the NIST framework and the single-step execution level of Table 2. As cloud and cyber-physical infrastructures grow more complex, static IR struggles to maintain responsiveness. As system scale and inter-service dependencies increase, incident patterns exhibit high variance and non-deterministic symptom propagation across telemetry channels. The resulting data streams are heterogeneous and high-dimensional, while the absence of automated coordination between detection and mitigation pipelines introduces latency in fault isolation and recovery.

Recent studies focus on extending incident-response automation beyond static rule execution toward adaptive and reasoning workflows. RCACopilot [47] represents an early large-scale implementation that integrates LLM-based reasoning into cloud incident management. It automates data collection through predefined handlers that aggregate metrics, logs, and traces according to alert type, and applies LLM summarization and chain-of-thought reasoning to infer root-cause categories with explanatory narratives. The system supports scope switching and handler modification during runtime, allowing analysts to adjust evidence collection dynamically as incidents evolve. While its reasoning loop concludes after root-cause prediction, RCACopilot shows how contextual feedback and multi-source correlation can enhance the responsiveness of enterprise incident management. Building on this foundation, IRCopilot [48] introduces a more autonomous and reflective architecture. It organizes response activities through a collaborative set of LLM agents, including Planner, Generator, Analyst, and Reflector, operating over a persistent incident response tree. This structure maintains historical state and task dependencies, enabling continuous plan revision, evidence evaluation, and self-correction throughout the incident lifecycle. It demonstrates how incident handling can evolve into a closed feedback loop that learns and adapts across reasoning cycles. These systems illustrate the trajectory from deterministic playbooks to intelligent, feedback-driven automation.

From the perspective of dynamic task characterization, modern incident response systems are examples of nearly every dimension of our definition. Feedback integration is central since both RCACopilot and IRCopilot continuously revise their reasoning based on real-time evidence. In RCA-

Copilot, feedback occurs through runtime scope adjustments and handler updates that redefine which telemetry sources are analyzed, allowing the reasoning process to react to shifting incident states. IRCopilot advances this by embedding a reasoning–action–reflection loop in which outputs from the Generator and Analyst components feed directly into the Reflector and Planner, forming a self-correcting cycle that re-evaluates hypotheses as new information arrives. Contextual adaptation is achieved through flexible orchestration structures—dynamic handlers in RCACopilot and the evolving Incident Response Tree in IRCopilot—that modify response strategies to accommodate emerging symptoms, conflicting alerts, or environmental changes. Both frameworks demonstrate multi-step reasoning, using chain-of-thought or tree-of-thought methods to explore causal paths, weigh alternative explanations, and iteratively refine conclusions instead of relying on static classification.

Autonomy is another defining property. RCACopilot operates end-to-end once triggered, automatically collecting data, conducting RCA, and generating natural-language explanations. IRCopilot extends autonomy further by enabling agents to plan, act, and reflect without human supervision, effectively emulating the decision-making rhythm of an expert response team. These behaviors are sustained through persistent memory and evolution over time. RCA-Copilot retains processed incidents as new demonstrations that inform future analyses, while IRCopilot's Incident Response Tree maintains contextual state across reasoning cycles, preventing information loss and facilitating long-term improvement. Collectively, these capabilities move incident response beyond reactive remediation toward continuous diagnostic intelligence, where systems learn from experience, adapt strategies in real time, and improve with every case handled. However, RCACopilot is limited by bounded reasoning depth, and IRCopilot is constrained by LLM context and external validation. Integrating reinforcement-based optimization, causal verification, and live telemetry feedback could enable the next generation of IR agents to close the remaining loop, achieving fully dynamic, self-evolving response ecosystems that align precisely with the principles of dynamic cyber-automation defined in this study.

4.7 Red team automation

Red teaming tasks are also dynamic. Red team tools often rely on scripted exploits and vulnerability scanners that execute predefined sequences without interpreting results or adapting to defenses. Once an attack script completes, the process ends—no reasoning, feedback, or re-planning. This static design limits adaptability to modern infrastructures that employ dynamic access controls and runtime isolation. As environments evolve, effective red-team automation must move from

fixed exploit replay toward reasoning-based agents capable of adjusting tactics and goals through continuous feedback.

Recent work demonstrates how Large Language Models can realize this transformation. PentestGPT [25] is a fully dynamic framework in which reasoning, execution, and feedback are tightly coupled. Its architecture coordinates multiple modules—a reasoning component for goal decomposition, a generation module for command synthesis, and a parsing mechanism for interpreting results—all connected through a persistent Pentesting Task Tree that records context and prior outcomes. When an exploit fails or produces unexpected results, the system updates its plan and regenerates new actions, maintaining continuity across iterations. In doing so, PentestGPT integrates feedback directly into its reasoning loop, adapts its tactics to environmental change, and conducts multi-step planning with a degree of autonomy that mirrors human operators. The preserved task tree functions as memory, allowing the system to learn from previous attempts and maintain situational awareness over time. This closed reasoning–action–reflection cycle establishes PentestGPT as one of the clearest examples of dynamic red-team automation.

By contrast, the Autoattacker [50] represents an intermediate stage in this evolution. It employs an LLM to infer exploit chains across interconnected vulnerabilities, reasoning about privilege relationships and software dependencies to identify feasible attack paths. The model performs sophisticated multi-step inference but operates in a static analytical setting. Once inference concludes, it does not verify results against a live environment or adjust its reasoning based on outcomes. Its strength lies in contextual analysis, which is the ability to understand how discrete vulnerabilities compose an exploit chain. But it lacks direct feedback integration and autonomous decision-making. In this sense, it qualifies as a semi-dynamic system that demonstrates the cognitive reasoning required for full automation but not yet the adaptive control loop that defines a truly dynamic process. Future extensions that couple this reasoning engine with real-time exploit validation or environment feedback would close that gap.

Although LLMs are currently used in the context of dynamic cyber automation tasks, there are still several research gaps to be addressed in the next few years. Our next section describes relevant opportunities and future directions for dynamic cyber automation tasks.

5 Challenges and future directions

While recent work has made great progress in applying LLMs to cybersecurity, as shown in Table 5 our study reveals several consistent limitations across domains that highlight where further progress is needed, particularly in support of dynamic,

multi-step task automation. These findings also directly motivate the future directions.

5.1 Privacy, data governance, and deployment constraints

Beyond model reliability, cybersecurity automation raises substantial privacy and data governance concerns. Dynamic cyber tasks may require access to system logs, endpoint telemetry, source code, asset inventories, identity records, network configurations, and incident reports. Disclosure of these data can reveal internal infrastructure, active vulnerabilities, defensive capabilities, or ongoing investigations. The risk extends beyond the initial prompt because tool using agents may propagate sensitive information through model inputs, external tools, intermediate outputs, memory, and generated reports [42]. Privacy should therefore be enforced across the complete workflow rather than treated only as a property of the underlying model.

Running models on premises can reduce the need to transmit internal data to an external provider and may support organization specific customization. However, local deployment does not eliminate security and governance requirements. It introduces costs related to computing resources, model maintenance, access control, auditing, and protection of the deployed model from unauthorized access or theft [51]. The choice between local and externally hosted inference should therefore depend on data sensitivity, operational capacity, and the consequences of disclosure. Practical controls include data minimization, role based access, input redaction, retention limits, and auditable records of the information supplied to the model.

RAG can provide access to current threat intelligence and internal organizational knowledge without embedding all sensitive information in model parameters. However, the retrieval layer creates its own privacy risks because private documents may be exposed through retrieval results, indexes, queries, or generated responses [52]. Cybersecurity systems should enforce authorization during both data ingestion and retrieval, isolate knowledge stores according to user and agent privileges, and limit the information retained in agent memory. Future work should evaluate privacy across the entire pipeline, including retrieval, tool invocation, memory, and output generation, while balancing confidentiality with the contextual information required for effective automation.

5.2 Threat and knowledge drift in cybersecurity

Cybersecurity presents a particularly demanding form of concept and knowledge drift because vulnerabilities, malware behaviors, attacker infrastructure, defensive controls, and exploitation techniques change continuously. Models trained on historical data may become less reliable as net-

work behavior and operational baselines evolve, a problem already observed in intrusion detection systems [53]. LLMs introduce an additional source of instability because their behavior may also change across model versions even when the same prompts are used [54]. More importantly, a model relying only on parametric knowledge cannot reliably reason about newly disclosed vulnerabilities, emerging attack campaigns, or zero-day threats that were absent from its training data. Previously valid recommendations may also become outdated when software versions, configurations, indicators of compromise, or mitigation guidance change.

Retrieval-Augmented Generation can reduce this limitation by grounding model reasoning in current vulnerability feeds, threat intelligence reports, asset inventories, security logs, and organization-specific procedures. Recent work demonstrates the potential of hybrid retrieval to improve cybersecurity knowledge retention, temporal reasoning, and adaptation to emerging threats [55]. However, retrieval does not remove the adaptation problem. It shifts the challenge toward constructing and maintaining a knowledge layer that is sufficiently current, complete, trustworthy, and accessible under organizational security policies. Public threat intelligence may be delayed, incomplete, duplicated, or contradictory, while local information such as software inventories, network topology, configurations, and active investigations may be sensitive and subject to strict access controls.

Cybersecurity knowledge graphs can provide structured relationships among vulnerabilities, products, indicators, tactics, mitigations, and affected assets. ThreatKG, for example, continuously integrates knowledge extracted from multiple open-source threat intelligence sources into an evolving threat graph [56]. More broadly, recent literature reflects growing but methodologically diverse interest in CTI knowledge graphs, including entity recognition, relation extraction, graph representation, and LLM-based construction methods [57]. Nevertheless, reusable knowledge graphs with broad, timely, and version-aware coverage remain difficult to maintain. TRACE identifies delayed incorporation of rapidly evolving unstructured information as a source of knowledge-graph hysteresis and highlights the need for continuous retrieval, entity extraction, and alignment across heterogeneous cybersecurity sources [58].

These limitations are especially important for zero-day and rapidly evolving threats. The delay between initial discovery, public disclosure, identifier assignment, threat-intelligence publication, local ingestion, and reliable retrieval may leave an LLM without the required information during the most critical period. Dynamic cyber systems should therefore treat knowledge freshness as an explicit system property rather than assuming that retrieval automatically provides current context. Retrieved evidence should include timestamps, source provenance, affected product versions, confidence levels, and expiration conditions. Sys-

tems should also detect stale or conflicting evidence, trigger re-analysis when new information becomes available, and restrict retrieval according to user and agent authorization.

Future work should evaluate not only whether a model eventually adapts, but also how quickly and reliably it incorporates new threat knowledge. Relevant measures include knowledge-ingestion latency, retrieval freshness, version-matching accuracy, contradiction resolution, and performance before and after a newly disclosed threat is added to the knowledge base. Continual-learning and memory-management methods may help balance adaptation with retention of previously useful knowledge [59], but they should be coordinated with retrieval, access-control, and provenance mechanisms across the complete cybersecurity workflow. This combination is necessary to ensure that dynamic cyber agents remain responsive to emerging threats without overreacting to temporary signals or relying on stale and unsupported information.

5.3 Operational utility, feedback validation, and human oversight

The practical value of LLM-based cybersecurity automation cannot be established solely through model accuracy or the apparent coherence of generated outputs. A system may produce plausible recommendations while increasing alert volume, requiring extensive verification, or introducing additional failure modes. The relevant question is therefore whether its use reduces the total effort required to reach a correct, timely, and defensible decision. Recent empirical work on security incident summarization shows that autonomous LLM outputs may omit critical information or introduce factual inaccuracies, while collaborative use can reduce analyst effort and improve report readability and consistency [60].

Operational utility also depends on whether feedback is grounded in observable security outcomes. Self-Refine [61] demonstrates how an LLM can iteratively critique and revise its output using self-generated textual feedback, but improved coherence does not establish that the revised output produces a better operational result. Pan et al. [62] further show that models in iterative loops may optimize for the feedback signal rather than the intended objective. Tool-interaction frameworks such as ReAct [63] provide external observations, but successful command execution or symbolic task completion does not necessarily mean that an incident was contained, a vulnerability was mitigated, or system integrity was restored.

Future systems should therefore connect reasoning updates to measurable changes in security state, such as whether an exploit path was blocked, an affected host was contained, service availability was restored, or threat exposure was reduced. Research on causal reinforcement learning indicates that feedback lacking valid cause-and-effect relationships

can reinforce spurious correlations and destabilize adaptation [64]. Human oversight remains necessary because these outcomes are often ambiguous and organization-dependent. Empirical evidence from security operations centres suggests that analysts primarily use LLMs for interpretation, context building, and technical communication while retaining authority over high-stakes decisions [65]. Deterministic policies, constrained action sets, and approval gates can further prevent generated actions from violating mandatory procedures or operational requirements [66].

Evaluation should consequently measure both feedback quality and net operational benefit. Relevant measures include time to triage, alerts processed per analyst, false escalation rate, recommendation acceptance rate, correction frequency, verification effort, and recovery time. An LLM that produces outputs quickly but requires extensive fact checking may provide little benefit and may become an additional burden. LLM-based automation should therefore aim to expand analyst capacity while preserving human accountability and grounding model adaptation in verifiable security outcomes. Future evaluations should also report end-to-end analyst time, inference and infrastructure costs, verification effort, and total operational workload so that the net benefit of LLM augmentation can be assessed.

5.4 Evaluation

Despite significant advances in evaluating LLMs for cybersecurity, most current benchmarks still capture static task accuracy rather than dynamic reasoning performance. Existing benchmarks provide valuable baselines for assessing domain knowledge and reasoning capability, but their evaluation designs remain inherently snapshot-based. Metrics like accuracy, F1-score, and ROUGE quantify correctness at a single inference step, assuming independent tasks with fixed contexts. CyberBench [67] evaluates ten cyber tasks through fixed textual datasets, while CyBench [68] broadens task coverage using multi-task evaluation but retains single-instance scoring. CS-Eval [69] introduces hierarchical levels of “knowledge,” “ability,” and “application,” but these tiers remain snapshot-based rather than iterative. This overlooks the temporal and feedback-driven nature of dynamic reasoning, where model outputs continuously influence subsequent states, observations, and decisions. As a result, improvements or regressions in reasoning across iterations are invisible, and models that over-adapt to noise or under-react to new evidence cannot be distinguished from genuinely adaptive agents. Evaluating LLMs in such non-deterministic settings is especially difficult because their reasoning paths are stochastic and context-sensitive. Identical prompts can yield different responses, which can complicate longitudinal measurement and reproducibility. These uncertainties make

it challenging to quantify genuine learning as opposed to incidental variance.

Future work should therefore move beyond static performance tables toward process-level evaluation frameworks that measure how models learn, adapt, and self-correct over time. One promising direction is to develop closed-loop testbeds where model outputs modify subsequent environmental inputs, enabling metrics such as adaptation latency to measure time to adjust after environmental changes, stability variance to measure performance fluctuation across feedback cycles, and cumulative improvement rate to measure net gain per reasoning iteration. Inspired by CS-Eval's multi-level design, future benchmarks could extend each level—knowledge, capability, application—into longitudinal tracks, where the same task reappears under altered conditions. Similarly, following CyberBench's multi-task modularity, benchmarks could embed inter-task dependencies, such as log analysis influencing vulnerability repair, to observe how models maintain reasoning consistency across domains. Integrating human-in-the-loop assessments, as already practiced in certain security operations, would further allow analysts to evaluate whether model adaptation aligns with operational expectations rather than superficial accuracy gains. In conclusion, dynamic evaluation should evolve from a static leaderboard paradigm to a longitudinal measurement of reasoning trajectories to capture how effectively a system stabilizes, recovers, and improves under continuous feedback.

5.5 Explainability

Dynamic reasoning frameworks have greatly advanced the autonomy of cyber agents, but their decision evolution remains largely opaque across continuous feedback cycles. Most systems describe what decision was reached but not how prior feedback reshaped later reasoning, leaving analysts unable to follow the agent's internal adjustments over time. This opacity is especially problematic in dynamic processes, where outputs, model states, and environmental conditions evolve together. When reasoning updates are applied repeatedly without transparent monitoring, small inconsistencies can compound into systemic drift. Recent studies confirm this risk that large language models that perform intrinsic self-correction frequently oscillate between answers or reinforce initial errors instead of converging, exposing unstable reasoning trajectories under feedback [70]. Empirical evidence also shows that even when LLMs attempt to explain themselves, their self-generated rationales do not faithfully reflect their internal causal reasoning. Huang et al. [71] find that explanations produced by instruction-tuned models are linguistically coherent but poorly correlated with true model behavior, often changing across prompts or feedback iterations. Likewise, the study of LLM explainability shows that current interpretation techniques focus on static snapshots—token

attributions, attention maps, or single-turn rationales, while offering little insight into multi-step or temporally dependent reasoning [72]. In dynamic reasoning contexts, this limitation becomes amplified: as agents continuously update under feedback, their latent reasoning shifts even when the textual explanations appear stable, creating a false impression of consistency. This decoupling between explanation and underlying reasoning trajectory further undermines transparency and complicates analyst oversight. Such limitations make it impossible to determine whether observed behavioral changes reflect genuine adaptation or accumulated bias. In multi-agent or continuously adaptive frameworks, feedback among agents introduces further opacity, which produces interdependent reasoning flows that are hard to understand afterwards. As a result, analysts cannot trace why an agent revised its understanding, verify whether adaptations align with evidence, or intervene before divergence occurs.

Future research should embed process-aware explainability directly into the feedback architecture of dynamic reasoning systems. Instead of treating explanations as static textual summaries, they must evolve alongside the agent's reasoning process, recording how environmental feedback, contextual changes, and internal updates interact over time. Promising directions include building dynamic reasoning traces that chronologically document decision transitions and their triggering evidence [73], as well as constructing causal provenance graphs that map how earlier reasoning influences later outcomes within continuous feedback loops [74]. Furthermore, integrating human-in-the-loop oversight would transform explainability from a diagnostic afterthought into a continuous safeguard.

5.6 Alternatives to LLM-based automation

LLMs are not the only mechanism available for automating cybersecurity workflows. Recommender systems can support incident handling by retrieving relevant information, ranking candidate response actions, and presenting context-sensitive recommendations to analysts [75]. Such systems may be particularly useful when organizations possess a sufficiently representative history of incidents and the response space can be expressed as a bounded set of alternatives. Compared with generative models, recommender systems can be simpler to operate and easier to validate because their outputs are selected from predefined or historically supported actions.

Rule-based and model-based playbooks provide another important alternative. Model-based incident response playbooks can encode response activities, dependencies, roles, and operational constraints in a machine-interpretable form [76]. Domain-specific playbooks can also provide adaptive automation without relying on open-ended language generation. For example, Anycast Agility uses network playbooks to

select suitable traffic-engineering responses to DDoS attacks according to attack conditions and network constraints [77]. More broadly, existing automatic incident response systems already support actions such as network isolation, policy reconfiguration, blocking, and system recovery using structured security signals and predefined response logic [78].

These alternatives may be more predictable, efficient, and usable than LLM-based agents when inputs are structured, procedures are stable, and the allowable action space is known in advance. Their limitations become more apparent when workflows involve heterogeneous unstructured evidence, previously unseen situations, ambiguous goals, or repeated reinterpretation of intermediate observations. In such settings, LLMs may provide value through natural language interpretation, cross-source synthesis, flexible planning, and tool coordination. A practical direction is therefore to combine these approaches in hybrid architectures, where recommender systems and playbooks constrain the available actions, while LLMs interpret unstructured context and support planning, interaction, and explanation.

6 Conclusion

This paper introduced a task-structure-oriented perspective that shifts focus from domain labels to the internal dynamics of cyber tasks. We defined the concept of dynamic cyber automation tasks, characterized by multi-step reasoning, temporal awareness, interaction with tools and environments, and the need to adapt decisions as contexts evolve. To organize this space, we developed a characterization framework grounded in four dimensions, including operational goals, knowledge-grounding requirements, collaboration modes, and cognitive complexity. We also showed how this structure provides a clearer lens for understanding what it means for an LLM to operate within real defense or attack workflows.

Using this framework, we examined LLM-enabled systems across seven major domains: threat intelligence, data privacy and security, vulnerability detection, malware detection, intrusion detection, incident response, and red-team automation. Across these domains, recent systems increasingly exhibit elements of dynamic behavior, including contextual integration, iterative reasoning, and partial autonomy. Frameworks such as PrivacyAsst, RCACopilot, IRCopilot, adaptive tracing systems, and pentesting agents demonstrate that LLMs can participate in feedback-driven workflows rather than serving solely as passive classifiers. However, our analysis also shows that these capabilities remain selective

and uneven. Many models still depend on static prompts, frozen retrieval pipelines, or single-cycle inference without maintaining state, memory, or causal grounding across iterations. Even systems using tool interaction or multi-agent architectures often lack mechanisms for stable adaptation, long-horizon reasoning, verification, or coordinated collaboration.

These observations motivated several future directions. First, dynamic cyber automation requires privacy-aware deployment and data governance mechanisms that protect sensitive operational information across retrieval, memory, tool interaction, and output generation. Second, systems must respond to threat and knowledge drift by maintaining timely, trustworthy, and access-controlled knowledge sources for newly disclosed vulnerabilities and emerging attacks. Third, feedback should be grounded in measurable security outcomes, while human analysts retain authority over ambiguous or high-impact decisions. Fourth, evaluation should move beyond snapshot accuracy toward process-level and operational measures that capture adaptation, verification effort, analyst workload, and recovery performance. Finally, LLMs should be considered alongside recommender systems, playbooks, and other established automation methods, with hybrid designs offering a promising balance between flexibility and operational control.

Overall, this study reframes LLM-enabled cybersecurity automation around how tasks unfold rather than only where they are applied. The proposed framework connects operational goals, knowledge-grounding requirements, collaboration modes, and cognitive complexity to the reasoning, feedback, and coordination required in dynamic cyber workflows. It also helps clarify where current systems remain limited, particularly in privacy, knowledge freshness, operational validation, and human oversight. Progress in this area will likely depend less on treating LLMs as standalone replacements for existing tools and more on integrating them with retrieval systems, playbooks, recommenders, and analyst decision making in controlled and auditable workflows.

A Detailed dynamic characteristics and limitations

This appendix provides the supporting evidence and key limitations associated with the representative tasks and systems classified in Table 5.

See Table 6.

Table 6 Supporting evidence of dynamic characteristics and key limitations of the representative LLM-enabled cyber tasks classified in Table 5

Task or system	Evidence of dynamic characteristics	Key limitation
TTP extraction from unstructured reports [40]	Uses contextual and retrieved evidence to map free-text CTI descriptions to MITRE ATT&CK tactics and procedures	The analysis relies largely on static retrieval and does not incorporate feedback-driven refinement across iterations
Cross-source CTI correlation and enrichment [41]	Continuously aggregates, correlates, and enriches indicators from heterogeneous sources such as honeypots, security logs, social media, and CERT feeds	The correlation process still relies on predefined rules and provides limited adaptation to evolving data sources or environmental feedback
Privacy-preserving tool orchestration [42]	Selects privacy-preserving tools according to task context and sensitivity and adjusts the transformation pipeline using intermediate tool outputs	User-side cryptographic operations introduce scalability and usability constraints for real-time adaptive workflows
Context-aware CVE triage [43]	Aligns natural-language CVE descriptions with asset and code context to prioritize potentially vulnerable functions or components	The current pipeline is primarily batch-oriented and lacks iterative tool feedback and continuous updates when assets or vulnerability information change
Post-exploitation remediation planning [44]	Revises attack and remediation plans based on reconnaissance results, exploitation outcomes, external vulnerability information, and tool feedback	Tool failures and environmental variability can propagate errors through the multi-agent workflow, while long reasoning chains may introduce hallucinations and context drift
TTP extraction and behavior modeling from interpreted malware [26]	Decomposes malware analysis into sequential reasoning steps that connect source code, metadata, behavioral hypotheses, and ATT&CK mappings	The zero-shot pipeline does not validate generated hypotheses through environmental feedback or iterative correction
Anomaly behavior-chain tracing [45, 46]	Adapts tracing or behavioral analysis according to evolving system events, temporal patterns, and user-specific activity context	The systems operate largely in isolation and provide limited contextual feedback, multi-agent coordination, and reasoning about broader operational behavior
Root-cause analysis from multi-source incident data [47, 48]	Combines logs, metrics, traces, planning, and reflection to revise evidence collection and root-cause hypotheses as an incident evolves	Some implementations depend on predefined handlers, remain constrained by context limits, and provide limited live coordination between tools and agents
Feedback-driven multi-step penetration testing [25]	Maintains a persistent task tree and updates attack plans after interpreting command results, failed actions, and changing environmental observations	Performance remains sensitive to parsing errors, long-horizon context loss, tool failures, and unexpected defensive responses

Author Contributions H.Y. and E.N.: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Validation, Writing—original draft. S.I.: Conceptualization, Formal analysis, Investigation, Project administration, Supervision, Writing—original draft. S.B.: Investigation, Supervision, Writing—review and editing. A.T.: Investigation, Supervision, Writing—review and editing.

Fundinfo Open access funding provided by the Defence Research and Development Canada (DRDC).

Data availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest Not applicable.

Ethical approval Not applicable.

Consent to participate Not applicable.

Consent for publication Not applicable.

Informed consent Not applicable.

Research involving human and/or animals Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Thirunavukarasu, A.J., Ting, D.S.J., Elangovan, K., Gutierrez, L., Tan, T.F., Ting, D.S.W.: Large language models in medicine. *Nat. Med.* **29**(8), 1930–1940 (2023)
2. Shu, D., Zhao, H., Liu, X., Demeter, D., Du, M., Zhang, Y.: LawLLM: law large language model for the us legal system. In: *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pp. 4882–4889 (2024)
3. Ko, H., Lee, J.: Can ChatGPT improve investment decisions? from a portfolio management perspective. *Financ. Res. Lett.* **64**, 105433 (2024)
4. Zhang, J., Bu, H., Wen, H., Liu, Y., Fei, H., Xi, R., Li, L., Yang, Y., Zhu, H., Meng, D.: When LLMs meet cybersecurity: a systematic literature review. *Cybersecurity* **8**(1), 55 (2025)
5. Ren, S., Chen, S.: Large language models for cybersecurity intelligence, threat hunting, and decision support. *Comput. Life* **13**(3), 39–47 (2025)
6. Liu, Z.: A review of advancements and applications of pre-trained language models in cybersecurity. In: *2024 12th International Symposium on Digital Forensics and Security (ISDFS)*, pp. 1–10 (2024)
7. Shi, D., Shen, T., Huang, Y., Li, Z., Leng, Y., Jin, R., Liu, C., Wu, X., Guo, Z., Yu, L., et al.: Large language model safety: a holistic survey. *arXiv preprint arXiv:2412.17686* (2024)
8. Hays, S., White, J.: Employing LLMs for Incident Response Planning and Review. *arXiv preprint arXiv:2403.01271* (2024)
9. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* **521**(7553), 436–444 (2015)
10. Hochreiter, S., Schmidhuber, J.: Long short-term memory. *Neural Comput.* **9**(8), 1735–1780 (1997)
11. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems*, vol. 30 (2017)
12. Peters, M.E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., Zettlemoyer, L.: Deep contextualized word representations. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 2227–2237. Association for Computational Linguistics, USA (2018)
13. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (long and Short Papers)*, pp. 4171–4186 (2019)
14. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* **21**(140), 1–67 (2020)
15. Radford, A., Narasimhan, K., Salimans, T., Sutskever, I.: Improving language understanding by generative pre-training (2018)
16. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al.: Training language models to follow instructions with human feedback. *Adv. Neural. Inf. Process. Syst.* **35**, 27730–27744 (2022)
17. Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F.L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S.: GPT-4 technical report (2023). *arXiv preprint arXiv:2303.08774*
18. Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H.W., Sutton, C., Gehrmann, S., et al.: PaLM: scaling language modeling with pathways. *J. Mach. Learn. Res.* **24**(240), 1–113 (2023)
19. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F.: LLaMA: open and efficient foundation language models, (2023). *arXiv preprint arXiv:2302.13971*
20. Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L.R., Lachaux, M.-A., Stock, P., Scao, T.L., Lavril, T., Wang, T., Lacroix, T., Sayed, W.E.: Mistral 7B (2023)
21. Team, G., Anil, R., Borgeaud, S., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A.M., Hauth, A., Millican, K.: Gemini: a family of highly capable multimodal models (2023). *arXiv preprint arXiv:2312.11805*
22. Ranade, P., Piplai, A., Joshi, A., Finin, T.: CyBERT: contextualized embeddings for the cybersecurity domain. In: *2021 IEEE International Conference on Big Data (Big Data)*, pp. 3334–3342 (2021)
23. Aghaei, E., Niu, X., Shadid, W., Al-Shaer, E.: SecureBERT: a domain-specific language model for cybersecurity (2022)
24. Liu, Z., Liao, Q., Gu, W., Gao, C.: Software vulnerability detection with GPT and in-context learning. In: *2023 8th International Conference on Data Science in Cyberspace (DSC)*, pp. 229–236. IEEE (2023)
25. Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., Rass, S.: PentestGPT: evaluating and harnessing large language models for automated penetration

- testing. In: 33rd USENIX Security Symposium (USENIX Security 24), pp. 847–864 (2024)
26. Zhang, Y., Zhou, X., Wen, H., Niu, W., Liu, J., Wang, H., Li, Q.: Tactics, techniques, and procedures (TTPs) in interpreted malware: a zero-shot generation with large language models (2024). arXiv preprint [arXiv:2407.08532](https://arxiv.org/abs/2407.08532)
 27. Zangana, H.M., Mustafa, F.M., Li, S.: Large language models in cybersecurity: from automation to intelligence. In: Leveraging Large Language Models for Quantum-Aware Cybersecurity, pp. 277–300. IGI Global Scientific Publishing, USA (2025)
 28. Sultana, M., Taylor, A., Li, L., Majumdar, S.: Towards evaluation and understanding of large language models for cyber operation automation. In: 2023 IEEE Conference on Communications and Network Security (CNS), pp. 1–6 (2023)
 29. Motlagh, F.N., Hajizadeh, M., Majd, M., Najafi, P., Cheng, F., Meinel, C.: Large language models in cybersecurity: State-of-the-art (2024). arXiv preprint [arXiv:2402.00891](https://arxiv.org/abs/2402.00891)
 30. Xu, H., Wang, S., Li, N., Wang, K., Zhao, Y., Chen, K., Yu, T., Liu, Y., Wang, H.: Large language models for cyber security: a systematic literature review. ACM Trans. Softw. Eng. Methodol. (2024). <https://doi.org/10.1145/3769676>
 31. Chen, Y., Cui, M., Wang, D., Cao, Y., Yang, P., Jiang, B., Lu, Z., Liu, B.: A survey of large language models for cyber threat detection. Comput. Secur. **145**, 104016 (2024)
 32. Ferrag, M.A., Alwahedi, F., Battah, A., Cherif, B., Mechri, A., Tihanyi, N.: Generative AI and large language models for cyber security: all insights you need. Available at SSRN 4853709 (2024)
 33. Gennari, J., Lau, S.-H., Perl, S., Parish, J., Sastry, G.: Considerations for evaluating large language models for cybersecurity tasks. SEI Insights 20 (2024)
 34. Hasanov, I., Virtanen, S., Hakkala, A., Isoaho, J.: Application of large language models in cybersecurity: a systematic literature review. IEEE Access **12**, 176751–176778 (2024)
 35. Hassanin, M., Moustafa, N.: A comprehensive overview of large language models (LLMs) for cyber defences: Opportunities and directions (2024). arXiv preprint [arXiv:2405.14487](https://arxiv.org/abs/2405.14487)
 36. Tian, S., Zhang, T., Liu, J., Wang, J., Wu, X., Zhu, X., Zhang, R., Zhang, W., Yuan, Z., Mao, S., et al.: Exploring the role of large language models in cybersecurity: a systematic survey. arXiv preprint [arXiv:2504.15622](https://arxiv.org/abs/2504.15622) (2025)
 37. Xu, M., Fan, J., Huang, X., Zhou, C., Kang, J., Niyato, D., Mao, S., Han, Z., Lam, K.-Y.: Forewarned is forearmed: a survey on large language model-based agents in autonomous cyberattacks (2025). arXiv preprint [arXiv:2505.12786](https://arxiv.org/abs/2505.12786)
 38. Manocchio, L.D., Layeghy, S., Lo, W.W., Kulatilleke, G.K., Sarhan, M., Portmann, M.: Flowtransformer: a transformer framework for flow-based network intrusion detection systems. Expert Syst. Appl. **241**, 122564 (2024)
 39. Pascoe, C.E.: Public draft: The NIST cybersecurity framework 2.0. National Institute of Standards and Technology (2023)
 40. Fayyazi, R., Taghdimi, R., Yang, S.J.: Advancing TTP analysis: harnessing the power of large language models with retrieval augmented generation. In: 2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops), pp. 255–261. IEEE (2024)
 41. Spyros, A., Koritsas, I., Papoutsis, A., Panagiotou, P., Chatzakou, D., Kavallieros, D., Tsirikas, T., Vrochidis, S., Kompatsiaris, I.: AI-based holistic framework for cyber threat intelligence management. IEEE Access **13**, 20820–20846 (2025)
 42. Zhang, X., Xu, H., Ba, Z., Wang, Z., Hong, Y., Liu, J., Qin, Z., Ren, K.: PrivacyAsst: safeguarding user privacy in tool-using large language model agents. IEEE Trans. Dependable Secure Comput. **21**, 5242–5258 (2024)
 43. Wu, Y., Wen, M., Yu, Z., Guo, X., Jin, H.: Effective vulnerable function identification based on CVE description empowered by large language models. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp. 393–405 (2024)
 44. Huang, J., Zhu, Q.: PenHeal: a two-stage LLM framework for automated pentesting and optimal remediation. In: Proceedings of the Workshop on Autonomous Cybersecurity, pp. 11–22 (2023)
 45. Darvishi, K., Noferesti, M., Ezzati-Jivan, N.: Toward adaptive tracing: efficient system behavior analysis using language models. In: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, pp. 62–66 (2024)
 46. Song, S., Zhang, Y., Gao, N.: Confront insider threat: Precise anomaly detection in behavior logs based on LLM fine-tuning. In: Proceedings of the 31st International Conference on Computational Linguistics, pp. 8589–8601. (2025)
 47. Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., et al.: Automatic root cause analysis via large language models for cloud incidents. In: Proceedings of the Nineteenth European Conference on Computer Systems, pp. 674–688 (2024)
 48. Lin, X., Zhang, J., Deng, G., Liu, T., Liu, X., Yang, C., Zhang, T., Guo, Q., Chen, R.: IRCopilot: Automated incident response with large language models. arXiv preprint [arXiv:2505.20945](https://arxiv.org/abs/2505.20945) (2025)
 49. Brezinski, K., Ferens, K.: Metamorphic malware and obfuscation: a survey of techniques, variants, and generation kits. Secur. Commun. Netw. **2023**(1), 8227751 (2023)
 50. Xu, J., Stokes, J.W., McDonald, G., Bai, X., Marshall, D., Wang, S., Swaminathan, A., Li, Z.: AutoAttacker: A large language model guided system to implement automatic cyber-attacks. arXiv preprint [arXiv:2403.01038](https://arxiv.org/abs/2403.01038) (2024)
 51. Huang, H., Li, Y., Jiang, B., Jiang, B., Liu, L., Liu, Z., Sun, R., Liang, S.: A middle path for on-premises LLM deployment: preserving privacy without sacrificing model confidentiality. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 8321–8359. Association for Computational Linguistics, China (2025)
 52. Zeng, S., Zhang, J., He, P., Xing, Y., Liu, Y., Xu, H., Ren, J., Wang, S., Yin, D., Chang, Y., Tang, J.: The good and the bad: exploring privacy issues in retrieval-augmented generation (RAG). In: Findings of the Association for Computational Linguistics: ACL 2024, pp. 4505–4524. Association for Computational Linguistics, Thailand (2024)
 53. Shyaa, M.A., Ibrahim, N.F., Zainol, Z., Abdullah, R., Anbar, M., Alzubaidi, L.: Evolving cybersecurity frontiers: a comprehensive survey on concept drift and feature dynamics aware machine and deep learning in intrusion detection systems. Eng. Appl. Artif. Intell. **137**, 109143 (2024)
 54. Chen, L., Zaharia, M., Zou, J.: How is ChatGPT's behavior changing over time? Harvard Data Science Review 6(2) (2024)
 55. Borah, A., Alam, M.T., Rastogi, N.: Adapting large language models to emerging cybersecurity using retrieval-augmented generation. (2025). arXiv preprint [arXiv:2510.27080](https://arxiv.org/abs/2510.27080)
 56. Gao, P., Liu, X., Choi, E., Ma, S., Yang, X., Song, D.: ThreatKG: an AI-powered system for automated open-source cyber threat intelligence gathering and management. In: Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis, pp. 1–12 (2024)
 57. Papoutsoglou, M., Meditskos, G., Bassiliades, N., Kontopoulos, E., Vrochidis, S.: Mapping the current status of CTI knowledge graphs through a bibliometric analysis. In: Proceedings of the 13th Hellenic Conference on Artificial Intelligence, pp. 1–6 (2024)
 58. Xu, Z., Ning, Z., Hu, T., Zhuge, J., Wang, Y., Cao, J., Xu, M.: TRACE: timely retrieval and alignment for cybersecurity knowledge graph construction and expansion. arXiv preprint [arXiv:2602.11211](https://arxiv.org/abs/2602.11211) (2026)

59. Ashrafee, A., Kozal, J., Wozniak, M., Krawczyk, B.: Holistic continual learning under concept drift with adaptive memory realignment (2025). arXiv preprint [arXiv:2507.02310](https://arxiv.org/abs/2507.02310)
60. Kramer, D., Rosique, L., Narotam, A., Bursztein, E., Kelley, P.G., Thomas, K., Woodruff, A.: Integrating large language models into security incident response. In: Twenty-First Symposium on Usable Privacy and Security (SOUPS 2025), pp. 133–148 (2025)
61. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y.: Self-refine: iterative refinement with self-feedback, p. 2023 (2023)
62. Pan, A., Jones, E., Jagadeesan, M., Steinhardt, J.: Feedback loops with language models drive in-context reward hacking, (2024). arXiv preprint [arXiv:2402.06627](https://arxiv.org/abs/2402.06627)
63. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y.: ReAct: synergizing reasoning and acting in language models. In: The Eleventh International Conference on Learning Representations (2022)
64. Zeng, Y., Cai, R., Sun, F., Huang, L., Hao, Z.: A survey on causal reinforcement learning. *IEEE Trans. Neural Netw. Learn. Syst.* **36**(4), 5942–5962 (2024)
65. Singh, R., Tariq, S., Jalalvand, F., Chhetri, M.B., Nepal, S., Paris, C., Lochner, M.: LLMs in the SOC: An empirical study of human-AI collaboration in security operations centres. In: 2026 IEEE Symposium on Security and Privacy (SP), pp. 4262–4281. IEEE (2026)
66. Barbieri, S., Meneses, L.V., Ferraz, Á.L.R., Júnior, L.A.P.: SOCpilot: Verifying policy compliance for LLM-assisted incident response. arXiv preprint [arXiv:2605.05501](https://arxiv.org/abs/2605.05501) (2026)
67. Liu, Z., Shi, J., Buford, J.F.: CyberBench: a multi-task benchmark for evaluating large language models in cybersecurity. *AAAI-24 Workshop on Artificial Intelligence for Cyber Security (AICS)* (2024)
68. Zhang, A.K., Perry, N., Dulepet, R., Ji, J., Menders, C., Lin, J.W., Jones, E., Hussein, G., Liu, S., Jasper, D., Peetathawatchai, P., Glenn, A., Sivashankar, V., Zamoshchin, D., Glikbarg, L., Askaryar, D., Yang, M., Zhang, T., Alluri, R., Tran, N., Sangpitsit, R., Yiorakdjis, P., Osele, K., Raghupathi, G., Boneh, D., Ho, D.E., Liang, P.: Cybench: a framework for evaluating cybersecurity capabilities and risks of language models. arXiv preprint [arXiv:2408.08926](https://arxiv.org/abs/2408.08926) (2024)
69. Yu, Z., Zeng, J., Chen, S., Xu, W., Xu, D., Liu, X., Ying, Z., Wang, N., Zhang, Y., Yang, M.: CS-Eval: A Comprehensive Large Language Model Benchmark for CyberSecurity. arXiv preprint [arXiv:2411.16239](https://arxiv.org/abs/2411.16239) (2025)
70. Zhang, Q., Wang, D., Qian, H., Li, Y., Zhang, T., Huang, M., Xu, K., Li, H., Liu, Y., Qiu, H.: Understanding the dark side of LLMs' intrinsic self-correction. arXiv preprint [arXiv:2412.14959](https://arxiv.org/abs/2412.14959) (2024)
71. Huang, S., Mamidanna, S., Jangam, S., Zhou, Y., Gilpin, L.H.: Can large language models explain themselves? a study of LLM-generated self-explanations (2023). arXiv preprint [arXiv:2310.11207](https://arxiv.org/abs/2310.11207)
72. Zhao, H., Chen, H., Yang, F., Liu, N., Deng, H., Cai, H., Wang, S., Yin, D., Du, M.: Explainability for large language models: a survey. *ACM Trans. Intell. Syst. Technol.* **15**(2), 1–38 (2024)
73. Yuan, C., Xie, Q., Huang, J., Ananiadou, S.: Back to the future: towards explainable temporal reasoning with large language models. In: Proceedings of the ACM Web Conference 2024, pp. 1963–1974 (2024)
74. Chi, H., Li, H., Yang, W., Liu, F., Lan, L., Ren, X., Liu, T., Han, B.: Unveiling causal reasoning in large language models: Reality or mirage? In: Advances in Neural Information Processing Systems, vol. 37, pp. 96640–96670 (2024)
75. Husák, M., Čermák, M.: SoK: applications and challenges of using recommender systems in cybersecurity incident handling and response. In: Proceedings of the 17th International Conference on Availability, Reliability and Security, pp. 1–10 (2022)
76. Shaked, A., Cherdantseva, Y., Burnap, P.: Model-based incident response playbooks. In: Proceedings of the 17th International Conference on Availability, Reliability and Security, pp. 1–7 (2022)
77. Rizvi, A., Bertholdo, L., Ceron, J., Heidemann, J.: Anycast agility: network playbooks to fight DDoS. In: 31st USENIX Security Symposium (USENIX Security 22), pp. 4201–4218 (2022)
78. Karlzen, H., Sommestad, T.: Automatic incident response solutions: a review of proposed solutions' input and output. In: Proceedings of the 18th International Conference on Availability, Reliability and Security, pp. 1–9 (2023)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.